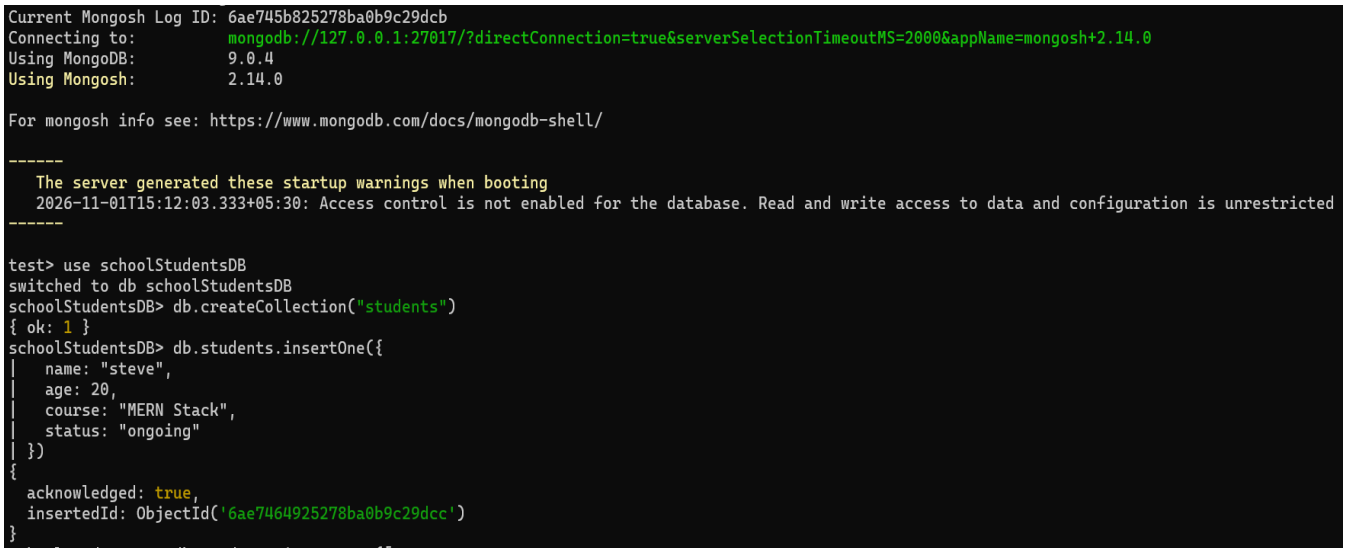
MongoDB Basic Practice Task – WEEK-12

Database and Collection Setup:

Step1: Select the database and create the collection

MongoDB database was selected as schoolStudentsDB, and a collection named students was created to store student details. One student record was also inserted successfully.



```
Current Mongosh Log ID: 6ae745b825278ba0b9c29dcb
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.14.0
Using MongoDB:          9.0.4
Using Mongosh:          2.14.0

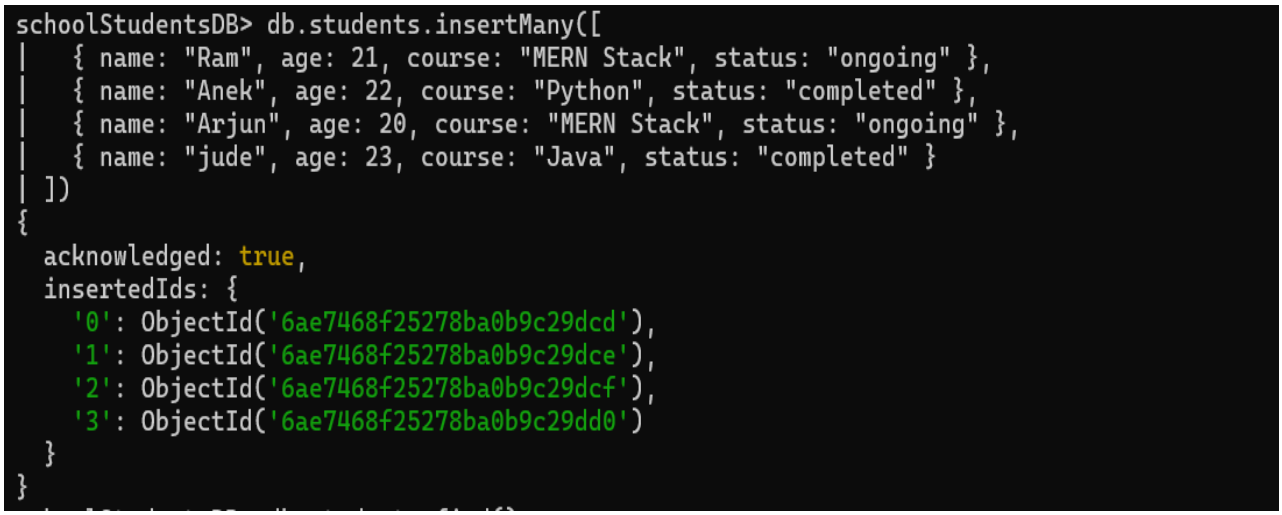
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-11-01T15:12:03.333+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------

test> use schoolStudentsDB
switched to db schoolStudentsDB
schoolStudentsDB> db.createCollection("students")
{ ok: 1 }
schoolStudentsDB> db.students.insertOne({
|   name: "steve",
|   age: 20,
|   course: "MERN Stack",
|   status: "ongoing"
| })
{
  acknowledged: true,
  insertedId: ObjectId('6ae7464925278ba0b9c29dcc')
}
```

Step 2: Insert Multiple Documents

The insertMany() command was used to add multiple student records with details such as name, age, course, and status. This helps us store several student records in the collection at once.



```
schoolStudentsDB> db.students.insertMany([
|   { name: "Ram", age: 21, course: "MERN Stack", status: "ongoing" },
|   { name: "Anek", age: 22, course: "Python", status: "completed" },
|   { name: "Arjun", age: 20, course: "MERN Stack", status: "ongoing" },
|   { name: "jude", age: 23, course: "Java", status: "completed" }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ae7468f25278ba0b9c29dcd'),
    '1': ObjectId('6ae7468f25278ba0b9c29dce'),
    '2': ObjectId('6ae7468f25278ba0b9c29dcf'),
    '3': ObjectId('6ae7468f25278ba0b9c29dd0')
  }
}
```

Step 3: Read Operations

Read operations help us view the student details stored in our collection. We can display all students or find students enrolled in a particular course.

Explanation: The find() command is used to retrieve student records. By adding a course condition, we can view only the students enrolled in MERN Stack.

```
schoolStudentsDB> db.students.find()
[
  {
    _id: ObjectId('6ae7464925278ba0b9c29dcc'),
    name: 'steve',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ae7468f25278ba0b9c29dcd'),
    name: 'Ram',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ae7468f25278ba0b9c29dce'),
    name: 'Anek',
    age: 22,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae7468f25278ba0b9c29dcf'),
    name: 'Arjun',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ae7468f25278ba0b9c29dd0'),
    name: 'jude',
    age: 23,
    course: 'Java',
    status: 'completed'
  }
]
schoolStudentsDB> db.students.find({ course: "MERN Stack" })
[
  {
    _id: ObjectId('6ae7464925278ba0b9c29dcc'),
    name: 'steve',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ae7468f25278ba0b9c29dcd'),
    name: 'Ram',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
```

Step 4: Update Operations

Update operations are used to change existing student details in the collection.

We can update one student's information or update multiple students at the same time.

```
schoolStudentsDB> db.students.updateOne(
|   { name: "steve" },
|   { $set: { status: "completed" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
schoolStudentsDB>
```

Explanation: updateOne() changes the first matching document, while updateMany() changes all matching documents. $set is used to change the specified field.

```
schoolStudentsDB> db.students.updateMany(
|   { course: "MERN Stack" },
|   { $set: { status: "completed" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 2,
  upsertedCount: 0
}
schoolStudentsDB>
```

Step 5: Delete Operations

Delete operations are used to remove documents from a collection. MongoDB provides commands to delete a single document or multiple documents.

```
schoolStudentsDB> db.students.deleteOne({ name: "Jude" })
{ acknowledged: true, deletedCount: 0 }
schoolStudentsDB>
```

```
schoolStudentsDB> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 5 }
schoolStudentsDB>
```

Explanation: deleteOne() removes the first matching document. deleteMany({}) removes all documents from the collection.

Step 6: Query Operators

Query operators help us search and filter student records based on different conditions.

A) $ gt – Greater than - Finds students older than 20.

```
schoolStudentsDB> db.students.find({ age: { $gt: 20 } })
[
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd2'),
    name: 'Ram',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd3'),
    name: 'Anek',
    age: 22,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd5'),
    name: 'Jude',
    age: 23,
    course: 'Java',
    status: 'completed'
  }
]
```

B). $lt – Less than -Finds students younger than 22.

```
schoolStudentsDB> db.students.find({ age: { $lt: 22 } })
[
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd1'),
    name: 'Steve',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd2'),
    name: 'Ram',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd4'),
    name: 'Arjun',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
```

C). $in – Matches values in a list

Finds students enrolled in either MERN Stack or Python.

```
schoolStudentsDB> db.students.find({
|   course: { $in: ["MERN Stack", "Python"] }
| })
[
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd1'),
    name: 'Steve',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd2'),
    name: 'Ram',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd3'),
    name: 'Anek',
    age: 22,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd4'),
    name: 'Arjun',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
```

D). $and – Matches both conditions - Finds students older than 20 whose status is completed.

```
schoolStudentsDB> db.students.find({
|   $and: [
|     { age: { $gt: 20 } },
|     { status: "completed" }
|   ]
| })
[
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd3'),
    name: 'Anek',
    age: 22,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd5'),
    name: 'Jude',
    age: 23,
    course: 'Java',
    status: 'completed'
  }
]
```

E). $or – Matches at least one condition - Finds students enrolled in Java or Python.

```
schoolStudentsDB> db.students.find({
|   $or: [
|     { course: "Java" },
|     { course: "Python" }
|   ]
| })
[
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd3'),
    name: 'Anek',
    age: 22,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd5'),
    name: 'Jude',
    age: 23,
    course: 'Java',
    status: 'completed'
  }
]
```

F). $exists – Checks whether a field exists - Finds student documents that contain the course field.

```
schoolStudentsDB> db.students.find({
|   course: { $exists: true }
| })
[
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd1'),
    name: 'Steve',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd2'),
    name: 'Ram',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd3'),
    name: 'Anek',
    age: 22,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd4'),
    name: 'Arjun',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ae74f3c25278ba0b9c29dd5'),
    name: 'Jude',
    age: 23,
    course: 'Java',
    status: 'completed'
  }
]
```

Real-World Use Case – Library System

A library system stores book details and helps manage book availability. We will practise inserting, updating, and searching book records.

1. Create a collection: db.createCollection("books")

```
schoolStudentsDB> db.createCollection("books")
{ ok: 1 }
schoolStudentsDB>
```

2. Insert books:

db.books.insertMany([

{ title: "Wings of Fire", author: "APJ Abdul Kalam", status: "available" },

{ title: "The Alchemist", author: "Paulo Coelho", status: "available" },

{ title: "Atomic Habits", author: "James Clear", status: "borrowed" }

])

```
schoolStudentsDB> db.books.insertMany([
|   { title: "Wings of Fire", author: "APJ Abdul Kalam", status: "available" },
|   { title: "The Alchemist", author: "Paulo Coelho", status: "available" },
|   { title: "Atomic Habits", author: "James Clear", status: "borrowed" }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ae750f825278ba0b9c29dd6'),
    '1': ObjectId('6ae750f825278ba0b9c29dd7'),
    '2': ObjectId('6ae750f825278ba0b9c29dd8')
  }
}
```

3. Update a book's status:

db.books.updateOne(

{ title: "Wings of Fire" },

{ $set: { status: "borrowed" } })

```
schoolStudentsDB> db.books.updateOne(
|   { title: "Wings of Fire" },
|   { $set: { status: "borrowed" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
schoolStudentsDB>
```

4. Search for available books:

db.books.find({ status: "available" })

```
schoolStudentsDB> db.books.find({ status: "available" })
[
  {
    _id: ObjectId('6ae750f825278ba0b9c29dd7'),
    title: 'The Alchemist',
    author: 'Paulo Coelho',
    status: 'available'
  }
]
```

Explanation: This library system stores book details, updates book availability, and searches for books that are available to borrow.

Conclusion

Through this task, I learned how to create databases and collections in MongoDB and perform basic CRUD operations. I also practised using query operators to search and filter data. The Library System example helped me understand how MongoDB can be used in real-world applications to store, update, and retrieve information.